

ME 350 F25 | Lab 10 Sensors/Microcontrollers 3: PID Control

Things to Complete Before Lab 10

Below is a list of things that each team should accomplish before attending Lab #10 in order to reduce frustration and maximize performance during final testing the following week. All tasks involve using the playing field in the X50 Assembly room. You have 24/7 access to this room and thus should be able to find a time to do this. Note: The X50 Assembly room and Mechatronics labs are reserved for use for ME450 design reviews during certain days. Refer to the ME350 announcements for details on these dates and times as during those times, only ME450 may use these rooms.

- 1. Make sure that your Flashlight mounting is properly adjusted so that the center of it is directly on the center of the targets**
- 2. Make sure you have your limit switches and hard stops set exactly where you want them to stop the mechanism at the leftmost and rightmost Targets.**
 - a. Why? You don't want to waste any time setting up.
 - b. How to accomplish this: Using the playing field in the X50 Assembly room, put the mechanism in the extreme positions and make sure that limit switches/hard stops are properly placed.
- 3. Mark all relevant positions on your machine using a Sharpie/scribe.**
 - a. Why: Quick identification of encoder counts
 - b. How to accomplish this: Using the playing field in the X50 Assembly room, put the mechanism in each position. Then mark these positions somehow on your machine using a sharpie or scribe or other method. In Lab 10, you will then easily be able to identify the proper encoder count for all positions via the Arduino "Encoder Count" sketch used in the last lab. (First, make sure the encoder count is zero, in your starting position.)
- 4. If you have not already, finish the last steps from the previous lab**

LAB 10: Sensors/Microcontroller 3 – PID Controllers

Purpose

In this lab you will learn the basics of PID feedback control. We will program a PID controller for the Arduino to position the flashlight precisely. You will extend on the lab setup from last week and find suitable proportional, derivative, and integral gains through tuning. You will then use the tuned PID controller to position your motor.

Motor Position Control Background

In control terms, the linkage, transmission, and motor are called the **plant** of your control system. This plant has a *dynamic* behavior. In reaction to an *input* signal (voltage sent to the motor), it will produce an *output*. In our case the output is the position of the flashlight, which is measured with the hall-effect encoder. The challenge with dynamic systems is that their output can change, even if you do not apply any input. A spring mass system, for example, will continue to oscillate for some time by itself after the spring has been deflected. In the case of our mechanism, the inertia of the linkage and gravity will create motion, even if the motor is switched off and no voltage is supplied. Because of this, we need to actively *control* the system to achieve the desired performance.

Let's look at the motor and linkage system to see how this can be done. As we learned earlier in the semester, for a DC motor, the motor torque T_m is proportional to the motor current i_m by a constant factor k_t . Rotation of the motor will produce a back-EMF voltage V_b that is proportional to the angular velocity ω of the shaft by the motor constant k_b . In SI units, the two constants are identical.

$$T_m = k_t i_m \quad V_b = k_b \omega \quad (1)$$

Where $k_t = k_b$ in SI units. We use Kirchhoff's voltage law to compute the motor current i . To make things simpler, we neglected the influence of the motor inductance. R is the electrical resistance in the motor coil, and V_s is the supply voltage:

$$T_m = k_t i_m = k_t \frac{V_s - V_b}{R} = k_t \frac{V_s - k_b \omega}{R} \quad (2)$$

So if we know ω and the desired motor torque T_m , we can compute V_s according to:

$$V_s = \frac{R}{k_t} T_m + k_b \omega \quad (3)$$

Furthermore, we can derive the following equations based on Newton's second law. In addition to the torque that is needed to accelerate the load, we apply a coulomb friction law that creates a constant force acting against the direction of motion and we include the torque generated by gravity. The motor torque T_m has to be equal to this load torque T_L .

$$T_L = J\dot{\omega} + T_f \cdot \text{sgn}(\omega) + T_{gravity} = T_m \quad (4)$$

Note: "sgn" is the signum or sign function which takes a value of ± 1 depending on the sign of ω .

In the equations above, J is the inertia of all the rotating parts, T_f is the coulomb friction torque of all joints and the transmission.

ME 350 F25 | Lab 10 Sensors/Microcontrollers 3: PID Control

In the previous lab, you have already identified the voltage values that are necessary to overcome friction and gravity. We can thus ignore these terms for the moment. We will compensate for them by adding the appropriate compensation voltages (as obtained in the last lab) at the end.

Our dynamic equations thus simplify to:

$$T_m = J\dot{\omega} = J\ddot{\theta} \quad (5)$$

Considering the voltage V_s as input and the angle θ as output in the motor position control case, we can build the block diagram of the system as presented below.

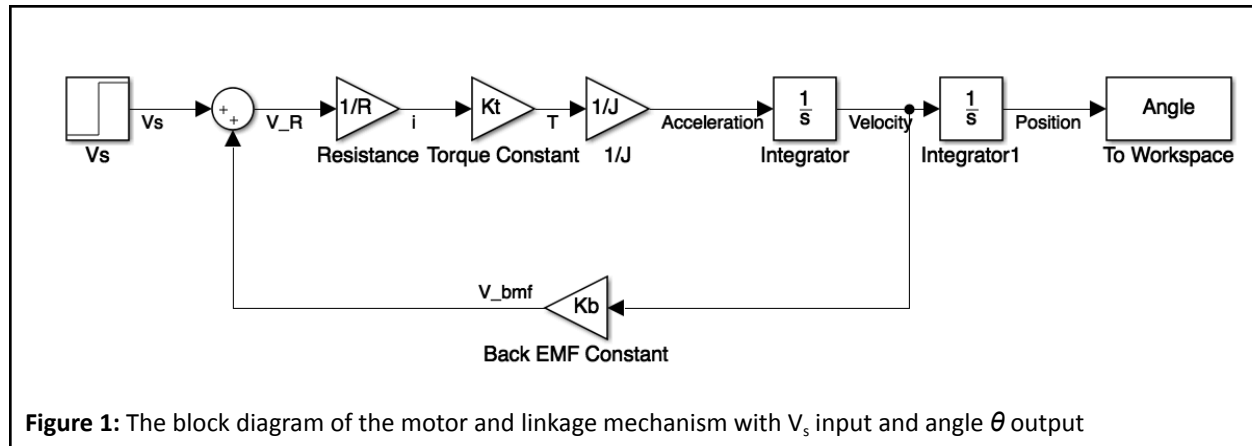
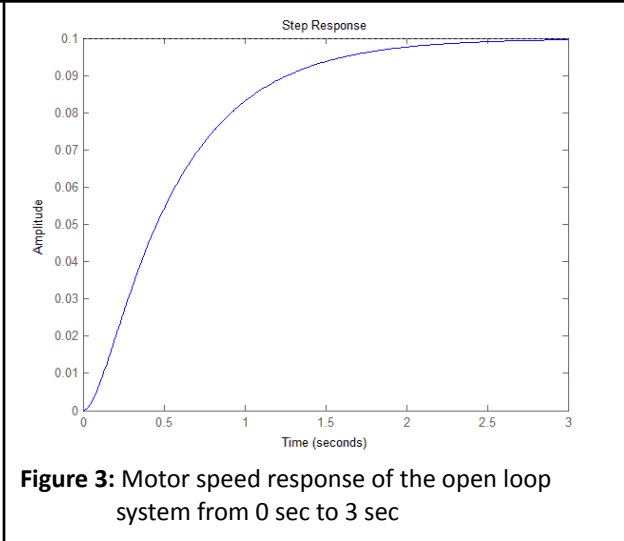
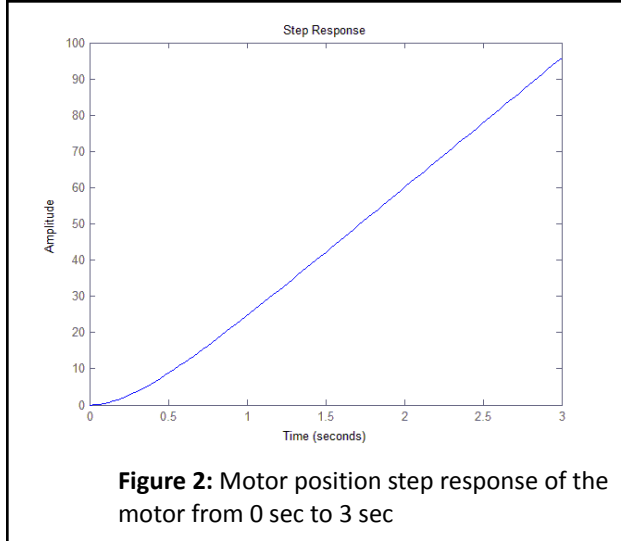


Figure 1: The block diagram of the motor and linkage mechanism with V_s input and angle θ output

If we would simply send a constant voltage command to the motor, this will create a current i and a constant motor torque T_m . You can imagine that this will cause the motor to accelerate. As the motor accelerates, the motor velocity ω goes up. This increases the back-EMF voltage, which reduces the voltage across the resistor and thus the motor current i . In other words, your motor torque is going down until it is equal to the load torque. At the intersection of the motor curve and the load curve, an equilibrium is reached. After a short time of acceleration, the motor will thus rotate with a constant speed. The motor angular position will keep increasing indefinitely. This *step response* of the output is shown below for position and velocity.

ME 350 F25 | Lab 10 Sensors/Microcontrollers 3: PID Control



This *open-loop* behavior of the system is certainly not what we want for the project. We want a position-controlled system that reliably moves to a desired position. To this end, we design a *closed-loop* feedback system with a proportional-integral-derivative (PID) controller. This is essentially the equivalent of a virtual spring/damper system (with an additional integral term) that pulls your flashlight in the right location. Let's ignore the integral term for a moment. The advantage of the remaining virtual spring/damper system over an actual physical spring/damper is that the neutral position of the spring (called the reference position $\theta_{reference}$) can be chosen to match the desired flashlight position (in the code this is called the target position).

To close the control loop, we have to measure the system output (the angle) and feed it back to the controller. The difference between the output and the reference value (where the flashlight should be) is determined as the control error $e = \theta_{reference} - \theta$. You can think of this as the deflection of your virtual spring. At the same time, we retrieve the velocity of the output and compare it to the reference velocity. This reference is 0, since the flashlight should be at rest once it is in the correct position. As a result, you get the derivative of the control error $\dot{e} = 0 - \dot{\theta}$. You can think of this as the compression velocity of your virtual damper.

Since we want the motor and linkage to behave like a spring damper system, we have to apply the appropriate torque. For a spring-damper system, the resulting torque would be:

$$T_m = C_{spring} (\theta_{reference} - \theta) + B_{spring} (0 - \dot{\theta}) \quad (6)$$

Using equation (3) derived above, we can directly solve for the desired motor voltage. Remember:

$$V_s = \frac{R}{k_t} T_m + k_b \omega = \frac{R}{k_t} T_m + k_b \dot{\theta}$$

and thus

ME 350 F25 | Lab 10 Sensors/Microcontrollers 3: PID Control

$$V_s = \frac{R}{k_t} \left[C_{spring} (\theta_{reference} - \theta) + B_{spring} (0 - \dot{\theta}) \right] + k_b \dot{\theta}$$

$$V_s = \frac{R \cdot C_{spring}}{k_t} (\theta_{reference} - \theta) + \left[\frac{R \cdot B_{spring}}{k_t} - k_b \right] (0 - \dot{\theta})$$

$$V_s = K_P (\theta_{reference} - \theta) + K_D (0 - \dot{\theta}) \quad (7)$$

We define two new constants: We refer to the scaled virtual spring constant $\frac{R \cdot C_{spring}}{K_t}$ as the proportional (P) gain K_P , and to the scaled virtual damper (which also includes the 'electrical' damping

due to K_b) $\frac{R \cdot B_{spring}}{K_t} - K_b$ as the derivative (D) gain K_D . Using these definitions, the control law can be written simply as:

$$V_s = K_P \cdot e + K_D \cdot \dot{e} \quad (8)$$

As discussed in class, we also include an integral term $K_I \int_0^t e dt$, which has no real analogous physical

device. It is necessary to compensate for steady state errors. Without this term, we potentially would see a steady state error, since equation (8) can only produce a voltage when an error e is present. If this was the case, the integral part would keep growing to apply an offset voltage which would only settle if $e=0$. This gives us the control equation:

$$V_s = K_P \cdot e + K_P \int e + K_D \cdot \dot{e} \quad (9)$$

In this lab, we will experimentally identify the controller parameters, K_P , K_I , and K_D . After using this equation to compute a V_s , we will simply add the voltages to compensate for friction and gravity (as they have been derived in the previous lab). The complete controller (including these compensation terms) is shown in figure 4.

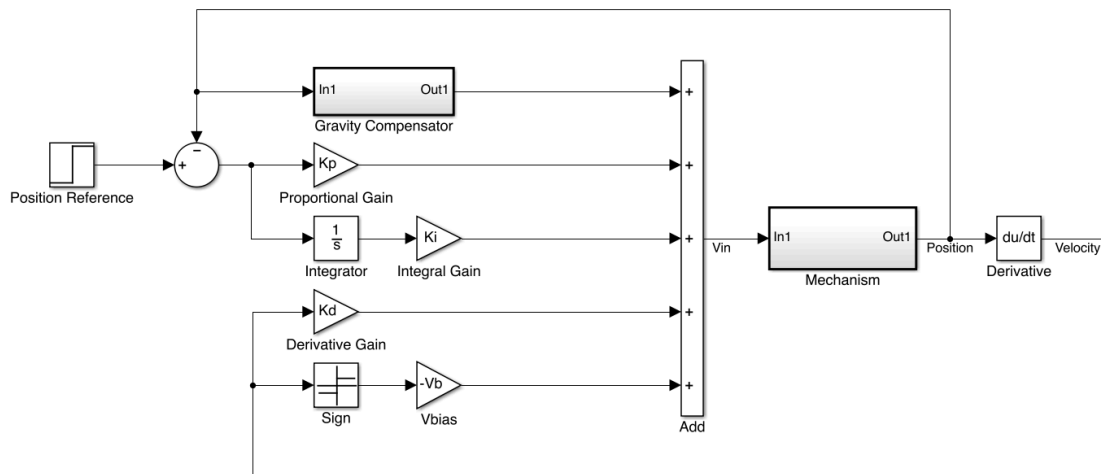


Figure 4: Closed loop control with compensation terms

ME 350 F25 | Lab 10 Sensors/Microcontrollers 3: PID Control

The task for today's lab is to find appropriate values for the controller gains K_p , K_i , and K_d . Let's start with K_p and K_d . If you remember that they are just scaled versions of spring and damper constants, it will be easier for you to find the right values. We can try different combinations of K_p and K_d and from the simulation result shown below (Figure 5), you can see that the final steady state θ reaches our desired reference value $\theta_{reference} = 1$ and is not drifting away. Also, as you may have noticed, the system responds faster as K_p increases, but it also fluctuates more and has larger overshoot. As K_d increases, it will bring less overshoot and fluctuations. If you think of the K_p gain as a spring, you can actually imagine that the object will be pulled faster if the spring has larger stiffness, corresponding to faster response for a larger K_p gain. Also, you may find it intuitive that the object connected to the spring normally oscillates more if the spring has larger stiffness. So how about the K_d gain? Let's think of it as damping. If you increase damping, the object usually will over-shoot less just as the effect when you increase the K_d gain. NOTE: If you increase the values of K_p and K_d too much, your system becomes so quick that the controller cannot react anymore. This will lead to an instability that you can see if your system starts to oscillate in an uncontrollable fashion. This is because K_p and K_d are only *virtual* springs and dampers. They react delayed and with limited forces, which is causing these instabilities. So DO NOT start with K_p and K_d values shown in Figure 5. The specific values depend on your mechanism. K_p in the Arduino code could have a magnitude of 0.1, with K_d usually one or two magnitudes smaller.

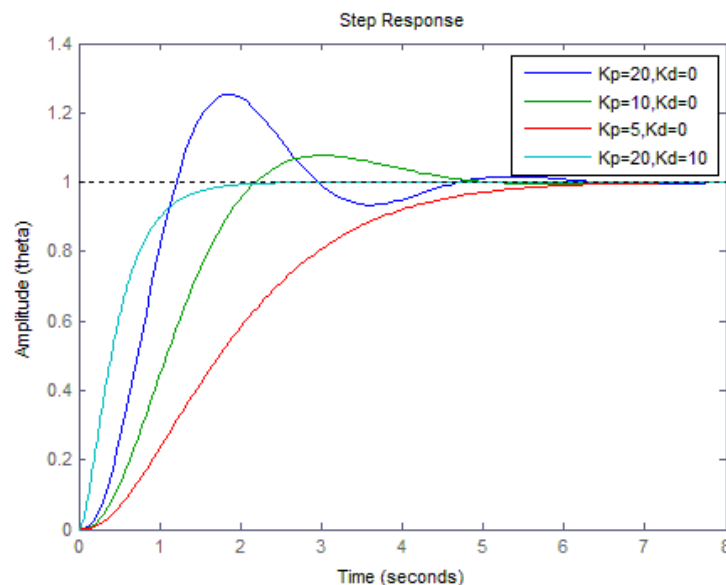


Figure 5: Closed loop response with different PID controllers

ME 350 F25 | Lab 10 Sensors/Microcontrollers 3: PID Control

Tuning Guideline

Knowing the basics about motor position control, you can start tuning the PID controller for your motor linkage system.

1. Download the [ME350_PID_Controller_TargetPractice.ino](#).
2. The parameter "activeTargetPosition" (shown in the blue box) can be set to move the mechanism to different target positions. You can pick any position to get started. Take target 2 position as an example. Change the line to:

```
int activeTargetPosition = TARGET_2_POSITION;
```

Follow the following paragraphs to tune the gains for this first position, then change activeTargetPosition to another target. Make sure your PID controller works at every position. If you observe significant overshoot you may need to retune the PID controller.

Note: make sure your power supply is on, and beware that your Arduino will believe whatever physical position your mechanism is at upon upload is encoder count zero; make sure to move your mechanism to your desired zero-position (i.e. all the way to the left).

3. Set the derivative gain "KD" and integral gain "KI" at 0 and gradually increase the "KP" starting from 0.01, until the system starts to overshoot or oscillate rapidly. This point depends a lot on your particular mechanism (weight, transmission ratio, friction, execution time for one loop of your code), but you can potentially go up to values of around 0.5. Be very careful in this process, since the overshoot or oscillations can seriously damage your linkage. Keep your hands out of the operating range and keep one finger on the power switch. If your linkage has reached its target position, you can use a pen to try moving it away and you will feel how the controller will apply a voltage to pull the mechanism back to its target position. The further you are away, the larger the voltage (and thus the torque) should be; just like with a real spring. Make sure that everybody in your team gets that experience and play around with lower KP values (softer spring) and higher KP values (stiffer spring). **Note the "KP" when the system overshoots significantly or oscillates continuously. Divide it by 2. This will be your upper limit for KP. Consider that you may also have to decrease FRICTION_COMP_VOLTAGE slightly now that KP is nonzero.**
4. Next, set the proportional gain "KP" and the integral gain "KI" to 0 and gradually increase the "KD" starting from 0.001, until the system starts to oscillate. Again, this point depends a lot on your particular mechanism (weight, transmission ratio, friction, execution time for one loop of your code), but you can potentially go up to values of around 0.1. Since KP is 0, your mechanism will not move by itself. Instead, you will have to use a pen to move it from side to side. You will notice that higher gains KD increase the "drag" that you will feel; just like with a real damper.

ME 350 F25 | Lab 10 Sensors/Microcontrollers 3: PID Control

The faster you try to move your linkage, the more torque it will create to oppose the motion. Again, make sure that everybody in your team gets that experience and play around with lower KD values (less damping) and higher KD values (more damping). DO NOT TOUCH THE MECHANISM DIRECTLY, ALWAYS USE A PEN. Keep one finger on the power switch. Note the "KD" when the system starts to oscillate. Divide it by 2. This will be your upper limit for KD.

5. Now you can play with the proportional gain and derivative gain to make sure that your system response is fast and has no overshoot. You might find that you want to use different values for different tasks. Play around with these values and gain experience. Make sure you stay within the limits established before.
6. Finally, even with non-zero FRICTION_COMP_VOLTAGE, KP, KD, and feed forward voltage, you may still find that your mechanism gets “stuck”. That is, it stops when it is not in the correct position. To remedy this, consider using a small KI (starting from 0.001). With a non-zero KI, the control system will slowly increase its voltage when it is not in the right position and can thus move the linkage from an incorrect “stuck” position to the desired position. Increasing KI too high can have undesirable effects. So be careful.

Typically, in control systems, the system performance can be evaluated with rise time, overshoot, and steady state error. The rise time is defined as the time required for the system output to rise from some lower level to some higher level of the final steady-state value. The typical range is 10%-90%. The steady state error is the difference between the reference value and the system output when the system is steady. The percent overshoot is the percent by which a system exceeds its final steady-state value. The rise time, steady-state error, and overshoot are presented in figure 7.

The proportional gain “KP” has the effect of reducing the rise time and will reduce but never eliminate the steady-state (S-S) error. The derivative gain “KD” ideally would have the effect of reducing overshoot and oscillation. The integral gain “KI” eliminates steady state error, but it could increase overshoot and lead to oscillations.

Play with your gains until you are satisfied with the performance. You might have to do this again in future labs, so it is more important that you get a feel for what each of the parameters does and in which range you can change them than finding the perfect value now. Write down the controller parameters that you found for each position.

ME 350 F25 | Lab 10 Sensors/Microcontrollers 3: PID Control

```
24 /** High-level behavior of the controller: */
25 // CONSTANTS:
26 // Target positions:
27 const int TARGET_1_POSITION = 0; // [encoder counts] Motor position corresponding to first target
28 const int TARGET_2_POSITION = 0; // [encoder counts] Motor position corresponding to second target
29 const int TARGET_3_POSITION = 0; // [encoder counts] Motor position corresponding to third target
30 const int TARGET_4_POSITION = 0; // [encoder counts] Motor position corresponding to fourth target
31 const int WAIT_POSITION = 0; // [encoder counts] Motor position corresponding to a wait position
32 const int LOWER_BOUND = TARGET_1_POSITION; // [encoder counts] Position of the left end stop
33 const int UPPER_BOUND = TARGET_4_POSITION; // [encoder counts] Position of the right end stop
34 const int TARGET_BAND = 5; // [encoder counts] "Close enough" range when moving towards a target.
35 // VARIABLES:
36 int activeTargetPosition = TARGET_3_POSITION; // [encoder counts] position of the currently active target
37
38 /** PID Controller */
39 // CONSTANTS:
40 const float KP = .1; // [Volt / encoder counts] P-Gain
41 const float KI = 0; // [Volt / (encoder counts * seconds)] I-Gain
42 const float KD = 0; // [Volt * seconds / encoder counts] D-Gain
43 const float SUPPLY_VOLTAGE = 10.0; // [Volt] Supply voltage at the HBridge
44 const float FRICTION_COMP_VOLTAGE = 0; // [Volt] Voltage needed to overcome friction
45 // VARIABLES:
```

Figure 6: Set target positions first. Using the values determined in the last lab, set FRICTION_COMP_VOLTAGE. Start by setting target band KP, KI, KD to 0. After that tune KP, KI, KD

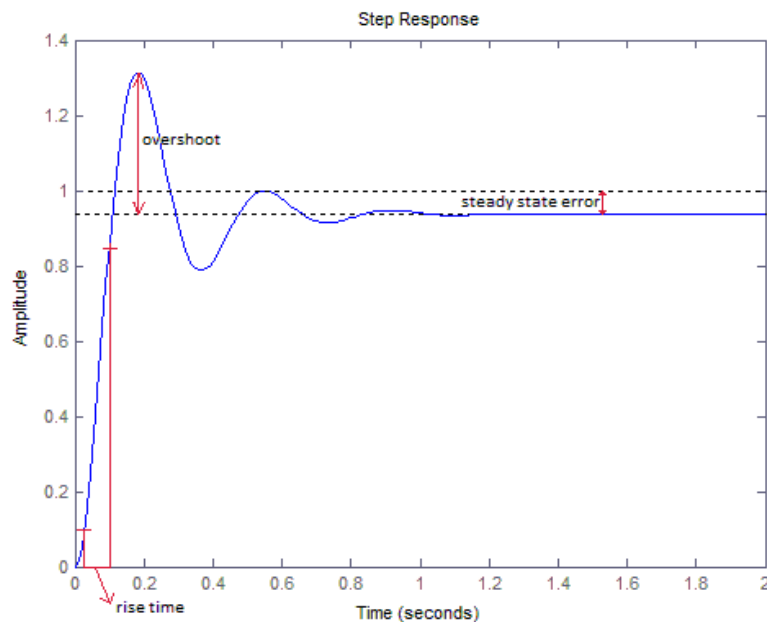


Figure 7: Illustration of rise time, steady state error and overshoot